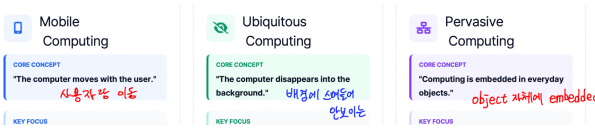
Mobile Computing



- wireless communication → intermittent connectivity (간헐적 연결)
  ↘ variable bandwidth
- adaptation → user context, network, resource availability

- SoC : CPU, GPU, NPU, ISP, DSP, SRAM
- dark silicon : 칩에 있는 모든 트랜지스터 발열 때문에 못쓴다!
  ⇒ TOPS/W 가중요

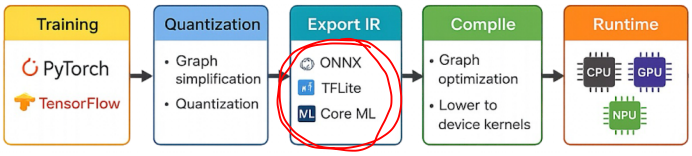
- throughput : 1초에 몇번 inference?
- tail latency : worst-case latency

Processor	Design Focus	Strength	Limitation	Role in On-Device AI
CPU	General-purpose control	• Flexible execution • Handles branching and irregular ops	Low energy efficiency for dense tensor math	• Orchestration • Pre/post-processing 전·후처리
GPU	Parallel throughput	• Massive parallel FP16 compute • High raw throughput	• Higher power consumption • Shared with graphics	• Medium-size models • Fallback execution
NPU	Energy-efficient tensor ops	• High TOPS per watt • Optimized for INT8 / INT4	Limited operator flexibility	Primary inference engine 주 inference

NPU → tensor 계산 energy efficient하게

- 1. Train model in PyTorch or TensorFlow
- 2. Prepare for deployment
  - graph simplification
  - quantization
  (모델 NPU가 좋아하게)
- 3. Export to an interchange format
  - ONNX
  - TFLite
  - Core ML format
  (IR 생성)
- 4. Compile for the target device
  - graph optimization
  - lowering to device kernels
  (IR을 device에 맞게 컴파일)
- 5. Run on device
  - runtime loads and executes
  - OS delegates to CPU, GPU, NPU
  - vendor drivers launch hardware

IR : Intermediate Representation (중간 형식)



- model compression의 goal 중에는 accuracy optimizes 있다!
- $q = \text{round}\left(\frac{r}{S}\right) + Z$ (r: real value, S: Scale (FP32), Z: zero-point (INT8))
- 보통 weight-only quant.

activation quat하면 accuracy 엄청 어려움.

- Post-Training Quant (PTQ) : calibration data로 range 추정 ⇒ retrain X
- Quantization-Aware Training (QAT)
  - forward 에서 fake quantization
  - backprop 에서 STE (straight-through estimator)
    : gradient 미분 없이
  - 모델이 학습 하면서 low precision 영향 미리 경험, 적응

- layer나 attention 마다 quant 하기 민감
  ⇒ mixed precision
  ex) FP16, INT8 섞어서

• fine-grained sparsity ⇒ 작은 weight 제거 (connection)

[w 0 w 0
 0 w w 0]

장 pruning granularity (세밀) → 높은 sparsity

단 irregular → 실제 hardware speedup ↓↓

단점.

• structured pruning : channel, filter, head, block, layer 단위 prune

- prune 기준
  - magnitude : 값자체 작은거
  - gradient : loss 에 영향 적은거
  - Hessian : 2차미분 → 제거했을때 loss 얼마나 증가 or 하는지
  - importance score

schedule
- one-shot
- iterative : 제거하면서 fine-tunning

• Distillation
- ground truth (hard) label : 정답만 알려줌
- teacher의 soft label : class간 유사도 (dark knowledge) 보여줌.
  ⇒ richer supervision

- Ground-truth labels
- Teacher predictions
- Objective: $L = \alpha L_{CE} + (1-\alpha) L_{KD}$
  - L_{CE}: cross-entropy with true labels
  - L_{KD}: distillation loss (difference between teacher and student outputs)

- logit, feature, attention 등 더 내부 표현까지 distill 하기도 함.

• Neural Architecture Search (NAS)

: search space 정하고 무엇을 최적화할지 정하면 아키텍처 찾아줌.

• Mamba

: attention 대신 SSM (State Space Model) 사용

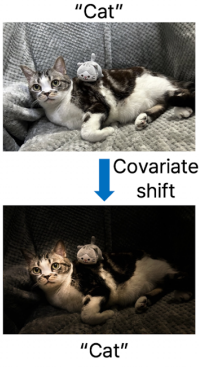
: 전체 sequence 다 안보고 압축된 hidden space만.

+ 선택적 update, delete

Model	Per-Step	Toal Compute	Memory
Transformer	$O(n^2)$	$O(n^3)$ total	O(1)
Transformer (with KV Cache)	O(n)	$O(n^2)$ total	O(n)
Mamba	O(1)	O(n) total	O(1)

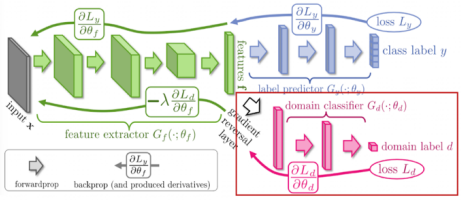
• Domain Shift (= Distribution Shift)

- **Covariate shift** (most common & our focus)
  - input distribution changes: *P(X) changes, P(Y|X) stays the same (X: input;Y: label)*
  - Example: indoor photos -> dark outdoor photos
  - often referred to as ***domain shift***
- **Label shift**
  - class prior changes: *P(Y) changes, P(X|Y) stays the same*
  - Example: weekday behavior -> weekend behavior
- **Concept shift**
  - input-label relationship changes: *P(Y|X) changes*
  - Example: same motion pattern, different phone placement
- **Temporal shift**
  - the distribution changes over time: *P_t(X, Y) changes over time*
  - Example: user routine changes over months
- In practice, on-device AI often faces several shifts at once



Unsupervised Domain Adaptation Method: DANN

- Ganin, Yaroslav, and Victor Lempitsky. "Unsupervised domain adaptation by backpropagation." International Conference on Machine Learning (ICML). 2015.



- Given "labeled source data" and "**unlabeled** target data",
- Feature extractor: extracts features from the input
- Label predictor: trains the features to be useful for predicting the class label } Typical DNN path
- Domain classifier: tries to distinguish whether the features come from the source or target domain
- Through the ***gradient reversal layer***, the feature extractor is trained to fool the domain classifier
- As the domain classifier gets better at separating source and target, the feature extractor learns features that make them **harder to distinguish** ↗ 도메인 차이 X ↗ 분류는 잘 해야
- The learned features become **invariant** across domains and **discriminative** for the task

15

source ⇒ labeled, target ⇒ unlabeled

※ source와 target을 구분 못하게 하는 feature학습

⇒ domain 차이 없애고, task 정보 유지

① feature extractor G_f : 입력으로부터 feature 추출] 일반적인 DNN

② label predictor G_y : feature로 class 예측

③ domain classifier : feature로 source or target 어디서온건지 구분

⇒ feature extractor가 domain classifier이 domain 구분못하게 망치도록 학습

⇒ feature가 비슷해짐.

- Adapter: 각 layer에 작은 module 넣고 이것만 학습.
  ⇒ bottleneck이나 기존 hidden representation modify
- Prefix tunning: attention에 virtual token 추가 → 학습 (input 아님)
- prompt tunning: input 앞에 학습된 virtual prompt token 추가 → 학습 (input level)
- LoRA: $W+\Delta W$ 에서 $\Delta W \approx AB$ low rank 두개로 근사 → 학습
- In-Context Learning: 학습 X, context에 정보 넣기
  - 장: immediate, no re-train
  - 단: context window, quality↓

- Common TTA mechanisms
  - entropy minimization (simple & common) → TENT
  - normalization-statistics update
  - self-supervised auxiliary objectives
  - pseudo-labeling
  - selective update of small parameter subsets

TTA현실문제
- 현실데이터 ⇒ 모델이 적응하고 overfit
- noise 있으면 확신가지고 학습
- 계속 update ⇒ on-device ↓↓
- batch 모으기 기다림.
- 여러번 추론 해야함

- Federated Learning 문제점
  - Statistical Heterogeneity: 사용자마다 data distribution 다름.
  - System Heterogeneity: 디바이스 성능이 다름.
  - Communication efficiency: 통신이 오히려 병목. (서버로 보내는)
  - privacy & security: 모델 update로 정보 유출 가능.

- split inference: model을 partition해서 intermediate feature 전달

- *LLM Execution Flow*
  - Text → Tokenizer → Text Embeddings → Transformer → Prediction
- *VLM Execution Flow*
  - Vision path: Image → Vision Encoder (e.g., CLIP, ViT) → Projector (e.g., MLP, Q-Former) → Visual Embeddings
    - **Vision Encoder** converts the input image into visual features, typically by splitting it into patches and encoding them into visual tokens
    - **Projector** maps visual features into the LLM token space (so that image and text can be processed together)
  - Text path: Text → Tokenizer → Text Embeddings ← 이건동일.
  - Common path: → Combined Embeddings → Transformer → Prediction